

Student tutorial: a CPU-friendly pipeline for top-quark tagging with classical ML and deep learning

BNBWU–CERN FYP collaboration
for BS Physics final-year students

May 28, 2026

Abstract

This is the practical companion to the research article. It walks you, step by step, through the codebase, the physics behind it, and exactly which commands to type to reproduce every figure. By the end you will (a) understand *what* the pipeline does and *why*, (b) be able to run it end-to-end on a laptop, and (c) be able to modify any of the four models for your own experiments.

Contents

1	What is this project, in one paragraph?	1
2	A bit of physics background	2
3	Project layout	2
4	Setting up the environment	3
4.1	macOS / Linux / WSL	3
4.2	Sanity check	3
5	Downloading the dataset	3
6	Running the headline pipeline	4
6.1	Stopping and resuming	4
7	Running the research-angle study	5
8	Reading the plots	5
9	How to modify the pipeline	6
10	Troubleshooting	6
11	Suggested student exercises	6

1 What is this project, in one paragraph?

At the LHC, a top quark almost always decays to a W boson and a b -quark; the W then decays to two more quarks. When the top is produced with high transverse momentum, all three of those quarks end up inside a single, large-radius jet. The job of a “top tagger” is to look at that jet and decide whether it really came from a top, or just from an ordinary QCD multi-parton splash. We compare four different ways to do that on a public benchmark dataset:

1. A **boosted decision tree** on hand-engineered jet features (the classical baseline).
2. A **convolutional neural network** on (η, ϕ) “jet images”.
3. **ParticleNet**, a graph neural network that treats the jet as an unordered cloud of particles.
4. The **Particle Transformer**, the modern self-attention architecture used in CMS/ATLAS.

The pipeline trains them, evaluates them, makes calibration plots and saves everything to `results/`. The research angle is in `run_study.py` and asks: *how much data does each model need, and are its scores trustworthy probabilities?*

2 A bit of physics background

The Standard Model, in 60 seconds

The Standard Model has six quarks, six leptons and four gauge bosons. The top quark is the heaviest, $m_t \simeq 172.76$ GeV. It is so heavy that it decays before it can hadronise: more than 99 % of the time, $t \rightarrow Wb$, and the W then decays either leptonically (to $\ell\nu$) or hadronically (to $q\bar{q}$). In this project we work with the *fully hadronic* case

$$t \rightarrow Wb \rightarrow (q\bar{q})b,$$

which gives three quarks inside one “boosted” jet at high p_T . The signature is a three-prong substructure and a jet mass $\sim m_t$. QCD jets are dominantly one-prong and lighter — this is the physical signal/background asymmetry we are exploiting.

Why “jet substructure”?

At hadron colliders one cluster final-state particles into “jets” using algorithms like anti- k_T . Once a single boosted jet contains the whole top decay, the *internal* structure of the jet (its mass, its N -subjettiness ratios τ_{21} , τ_{32} , its energy correlation functions C_2) becomes the discriminating information. The BDT baseline of this project uses exactly those hand-engineered features.

Why deep learning, then?

Hand-engineered features compress all the constituent information into 10–20 numbers. A deep network is allowed to look at all the constituent four-momenta and *learn* its own features. The question of this project is how much extra discrimination that freedom buys — and at what cost in training data and calibration.

3 Project layout

Listing 1: Top-level directory tree.

```
FYP_particle_physics/
+- config.py           all hyperparameters & paths
+- download_data.py    Zenodo fetcher
+- run_full_pipeline.py headline run (all 4 models)
+- run_study.py        data-efficiency + calibration sweep
+- requirements.txt
+- src/
| +- data_utils.py     HDF5 reader, feature engineering
| +- jet_cnn.py        CNN on jet images
| +- particle_net.py   EdgeConv graph network
| +- particle_transformer.py self-attention model
```

+- trainer.py	unified train/val/predict + BDT
+- evaluation.py	metrics + all publication plots
+- study_utils.py	data-efficiency + calibration plots
+ notebooks/	exploratory analysis
+ docs/	this tutorial + article + thesis
+ data/	created by download_data.py
+ results/	created at runtime

Tip

If you have never read this code before, the best entry points are `run_full_pipeline.py` (orchestrator), `config.py` (everything you can tune), and the `compute_*_features` functions in `src/data_utils.py` (the physics).

4 Setting up the environment

4.1 macOS / Linux / WSL

```
# 1) clone / cd into the project
cd FYP_particle_physics

# 2) create a fresh virtual environment
python3 -m venv particle_venv
source particle_venv/bin/activate

# 3) upgrade pip and install dependencies
pip install --upgrade pip
pip install -r requirements.txt
```

Watch out

Do not place the venv on a OneDrive or iCloud folder. Cloud synchronisation will mark large binary files (e.g. `libtorch_cpu.dylib`) as “online-only” placeholders, and the operating system will then fail to memory-map them, causing PyTorch to silently break at import time. Use a local, non-synced folder.

4.2 Sanity check

```
python3 -c "import torch, xgboost, h5py; print(torch.__version__)"
```

If this prints a torch version, you are good.

5 Downloading the dataset

```
python3 download_data.py      # answer 'y' at the prompt
```

This grabs three HDF5 files (about 1.6 GB total) from Zenodo and drops them into `data/`. The script is resumable: if your connection drops, just re-run it. If automatic download fails (e.g. proxy), download manually from zenodo.org/records/2603256 and place `train.h5`, `val.h5`, `test.h5` into `data/`.

6 Running the headline pipeline

The full headline run is a single command:

```
python3 run_full_pipeline.py
```

That will:

1. Read 5×10^4 jets per split from the HDF5 files.
2. Compute particle-level and jet-level features, plus jet images.
3. Train BDT, CNN, ParticleNet, Particle Transformer one after the other.
4. Save checkpoints to **results/models/**.
5. Generate all publication plots in **results/plots/** and a comparison table at **results/model_comparison.json**.

Useful flags:

```
python3 run_full_pipeline.py --model bdt           # only BDT
python3 run_full_pipeline.py --model cnn           # only CNN
python3 run_full_pipeline.py --model particlenet   # only ParticleNet
python3 run_full_pipeline.py --model transformer   # only Part
python3 run_full_pipeline.py --eval-only           # rebuild plots
                                                    # from saved ckpts
python3 run_full_pipeline.py --max-particles 50    # faster
```

Tip

The first time you run the pipeline, allow up to one hour on a laptop CPU. Subsequent runs in `-eval-only` mode regenerate plots from saved checkpoints in ~ 2 minutes.

6.1 Stopping and resuming

The pipeline is fully checkpointed. You can Ctrl-C at any moment — between epochs, in the middle of a batch, even by closing the laptop — and pick up exactly where you stopped by re-running the same command:

```
python3 run_full_pipeline.py           # starts (or resumes)
# ... Ctrl-C ...
python3 run_full_pipeline.py           # resumes mid-CNN, then
                                       # continues with ParticleNet
                                       # and Particle Transformer
```

What is persisted as you go:

- **results/models/<model>_last.pt**: full training state (model, optimiser, scheduler, AMP scaler, epoch counter, history) snapshot after every epoch.
- **results/models/<model>_best.pt**: best-val-AUC checkpoint, used for the final test prediction and the plots.
- **results/models/<model>_done.json**: a small marker file written when training for that model has *finished* (either max-epochs or early-stop). Models whose marker exists are skipped on the next run.
- **results/predictions/<model>_test.npz**: cached test predictions, used to skip the prediction step too.

- `results/<model>_history.json`: epoch-by-epoch loss and AUC, updated every epoch (so the training-history plot reflects partial progress as well).

To force a fresh re-train, pass `--force` (or delete `results/models/`). The same scheme applies to `run_study.py`: each (model, N_{train}) cell is resumable independently.

7 Running the research-angle study

Once the headline run finishes, the research contribution comes from

```
python3 run_study.py
```

This re-trains each model at $N_{\text{train}} \in \{5\,000, 10\,000, 25\,000, 50\,000\}$ jets, measures test AUC and background rejection at fixed signal efficiency, and computes the probability calibration of each model with and without a single-parameter temperature rescaling.

Selected flags:

```
# pick the sizes to sweep
python3 run_study.py --sizes 5000 25000 50000

# only some models
python3 run_study.py --models bdt cnn

# cap epochs per training (default 15)
python3 run_study.py --epochs-cap 10
```

8 Reading the plots

All plots land in `results/plots/`, in both PDF and PNG. The intent of each:

`jet_substructure.pdf` Signal/background distributions of all 13 engineered observables. The *physics check* – if this looks wrong, the dataset loading is broken.

`average_jet_images.pdf` The mean p_T -weighted image for top vs. QCD. You should *see* the three-prong topology in the difference panel.

`roc_comparison.pdf` ROC and background rejection vs. signal efficiency. The headline result.

`score_distributions.pdf` Per-model classifier output distributions. Look for the characteristic “peak at 0 / peak at 1” shape that well-trained binary classifiers produce.

`confusion_matrices.pdf` At threshold 0.5. Off-diagonal entries are misclassifications.

`training_history.pdf` Train/val loss and AUC vs. epoch. Use it to diagnose overfitting (train loss falling while val loss climbing) and to set the early-stop patience.

`bdt_importance.pdf` XGBoost gain-based feature importance. You should see jet mass, τ_{32} and constituent count among the top three.

`attention_maps.pdf` / `cls_attention.pdf` Particle Transformer interpretability.

`learning_curves.pdf` Output of `run_study.py`: AUC and rejection vs. training-set size.

`calibration.pdf` Reliability diagrams and ECE before/after temperature rescaling.

9 How to modify the pipeline

Try a new dataset subset size. Edit `config.py`: change `SUBSET_TRAIN / SUBSET_VAL / SUBSET_TEST`.

Try a deeper CNN. Edit `CNN_CONFIG["filters"]` in `config.py`, or the residual structure in `src/jet_cnn.py`.

Change the kNN of ParticleNet. `PARTICLENET_CONFIG["k_neighbors"]` in `config.py`.

Add more pairwise features for the Transformer. The pairwise features are computed in `compute_pairwise_features_batch` inside `src/data_utils.py`. Add another channel there (e.g. the invariant mass m_{ij}) and bump `PARTFORMER_CONFIG["pair_features"]` accordingly.

Add a new model.

1. Create a new file `src/my_model.py` with a PyTorch class implementing `forward(features, mask)` (and `count_parameters`).
2. Register a hyperparameter dict in `config.py`.
3. Add a new branch in `run_full_pipeline.py` following the `ParticleNet` branch as a template.

10 Troubleshooting

`ImportError: dlopen ... libtorch_cpu.dylib` The venv is on a OneDrive/iCloud-synced path. Re-create it on a local, non-synced folder (see Section 4).

`zsh: segmentation fault python3 run_full_pipeline.py` macOS OpenMP collision between NumPy's `libiomp5` and XGBoost's `libomp`. `run_full_pipeline.py` already sets `KMP_DUPLICATE_LIB_OK=TRUE` and pins XGBoost to one thread; if it persists, export `OMP_NUM_THREADS=1` in your shell as well.

`TypeError: a bytes-like object is required, not 'str' on pd.read_hdf` This pandas/PyTables bug is patched in `src/data_utils.py`: it reads HDF5 directly with PyTables and decodes block-column metadata by hand.

`FileNotFoundError: data/train.h5` Run `python3 download_data.py`.

`Killed / OOM during data prep` Lower the `SUBSET_*` values in `config.py`, or pass `-max-particles 50`.

11 Suggested student exercises

1. **Mass-decorrelation.** Modify the BDT to mask out the `jet_mass` feature. By how much does AUC drop? Plot the new ROC against the old one.
2. **Robustness to constituent count.** Restrict `-max-particles` to 20, 50, 100. How does the Transformer scale compared to ParticleNet?
3. **Try a fifth model.** Implement a simple Deep Sets baseline (per-particle MLP + sum pooling) and add it to the learning-curve plot.

4. **Re-do calibration with isotonic regression.** Implement post-hoc isotonic calibration (using `sklearn.isotonic.IsotonicRegression`) and compare its ECE against the temperature rescaling.
5. **Pile-up robustness.** Smear constituent p_T 's with a Gaussian kernel and re-evaluate. Which model degrades least?

Further reading

- Kasieczka *et al.*, “The Machine Learning landscape of top taggers”, SciPost Phys. 7 (2019) 014 — benchmark survey.
- Qu and Gouskos, “ParticleNet: Jet Tagging via Particle Clouds”, Phys. Rev. D 101 (2020) 056019.
- Qu, Li and Qian, “Particle Transformer for Jet Tagging”, ICML 2022, arXiv:2202.03772.
- Guo *et al.*, “On Calibration of Modern Neural Networks”, ICML 2017, arXiv:1706.04599.